ArcadeDB: A Native Multi-Model Embedded Database for Unified SQL, Graph, and Vector Workloads

Taewoon Kim
HumemAI
taewoon@humem.ai

Roberto Franchini
Arcade Data Ltd

Christian Himpe
University of Münster

Luca Garulli
Arcade Data Ltd

ABSTRACT

Modern applications increasingly combine relational analytics, graph traversals, and semantic vector retrieval, yet these workloads are typically split across multiple engines with duplicated data and complex pipelines. We present ArcadeDB, a native embedded multi-model database that executes SQL, Cypher, and HNSW-based vector search over a unified data representation. Our evaluation follows four pillars: table workloads (SQL OLTP / OLAP), graph workloads (Cypher OLTP / OLAP), vector indexing and search, and hybrid queries that compose all three modalities in a single workflow. On the Stack Overflow benchmark, we compare ArcadeDB against specialized baselines for each modality and report execution time, latency, storage footprint, and vector effectiveness metrics. The results highlight practical trade-offs and show that ArcadeDB enables end-to-end multi-model workflows without external orchestration while remaining competitive across individual workload classes.

PVLDB Reference Format:
Taewoon Kim, Roberto Franchini, Christian Himpe, and Luca Garulli. ArcadeDB: A Native Multi-Model Embedded Database for Unified SQL, Graph, and Vector Workloads. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

1 INTRODUCTION

Data systems for modern applications increasingly span multiple access patterns: transactional updates on structured records, relationship-centric traversals, and semantic retrieval over dense embeddings [8, 13, 20]. In practice, these patterns are often implemented as separate SQL, graph, and vector services connected through ETL pipelines, replication, and application-level orchestration [30, 31]. While this decomposition can optimize each subsystem in isolation, it also introduces substantial operational and development overhead: duplicated storage, cross-system consistency concerns, fragile synchronization pipelines, and higher end-to-end latency for composite queries.

This paper explores an alternative design point: a native embedded multi-model engine that supports relational, graph, and vector workloads over a unified data representation [10, 12, 13]. We present ArcadeDB as a system that executes SQL, Cypher, and HNSW-based vector retrieval in one runtime, enabling developers to compose hybrid workflows without exporting data across specialized engines. The key question is not whether one engine can dominate every single-model benchmark, but whether a unified architecture can remain competitive across workload classes while simplifying end-to-end application design. ArcadeDB is open-source software released under Apache-2.0, with source code and release artifacts publicly available at the official project repository [3]. The project also ships operational assets used in practice-oriented evaluation workflows (e.g., Docker-based runs and Kubernetes deployment assets), enabling independent reproduction in common engineering environments.

To answer this question, we evaluate ArcadeDB across four pillars derived from one consistent Stack Overflow data pipeline: (i) table workloads in SQL (OLTP and OLAP), (ii) graph workloads in Cypher (OLTP and OLAP), (iii) vector indexing and search with HNSW [14], and (iv) hybrid queries that combine SQL, graph traversals, and vector similarity in a single workflow. We compare against modality-appropriate baselines, including embedded relational systems [7, 26], a graph baseline, and four vector backends [9, 28, 29]. Our study emphasizes metrics that matter for practitioners: latency and throughput for single-threaded transactional paths (OLTP in this draft), analytical execution time under multi-threaded OLAP across all compared systems, index-build cost, storage footprint, and vector effectiveness.

Our contributions are threefold. First, we provide a unified evaluation methodology that measures SQL, graph, vector, and hybrid workloads under a shared dataset and lifecycle, extending prior multi-model benchmarking discussions [10, 31, 32]. Second, we present empirical results that characterize where a native multi-model embedded design is competitive and where trade-offs appear relative to specialized systems. Third, we demonstrate that integrated hybrid querying can be evaluated as a first-class capability rather than as an external orchestration pattern, and we release an executable open-source benchmark artifact so practitioners can rerun and adapt the workflow [3].

2 SYSTEM OVERVIEW

ArcadeDB is implemented as a native Java engine centered on `LocalDatabase`, a thread-safe embedded database instance that can be shared across worker threads. The runtime combines page-based storage, schema/type management, indexing, transaction control, and query execution in one process, following a unified multi-model design point [8, 13]. At the

storage layer, records and index structures are persisted as local components managed by the engine and coordinated with write-ahead logging (WAL) and configurable flush behavior. Transaction semantics are exposed through explicit begin/commit/rollback operations and isolation-level control, so application code and query execution share the same ACID boundary.

On top of this core, ArcadeDB uses a pluggable query-engine architecture. The database API accepts a language tag together with a query/command string, and dispatches execution through `QueryEngineManager`. SQL is provided by a native SQL engine, while OpenCypher is exposed through a dedicated Cypher engine with statement and plan caches [15, 19]. Additional engines (e.g., Gremlin/Mongo/GraphQL) can be registered at runtime when present on the classpath, allowing one storage/runtime core to support multiple front-end languages. This design is central for our study: SQL and Cypher operations are executed against the same underlying records, indexes, and transaction manager rather than synchronized copies across separate systems.

Graph processing is implemented in the Java core through `GraphEngine`, which manages vertex/edge creation and adjacency structures as first-class database data. Vector retrieval is implemented through `LSMVectorIndex`, a page-based transactional vector index integrated with ArcadeDB storage and configured via schema-level builders (dimensions, similarity function, HNSW connectivity/beam parameters, and quantization options) [14]. The vector stack also includes SQL-level vector functions, including hybrid scoring primitives that combine vector and keyword scores. Together, these components provide the architectural basis for the four-pillar evaluation in this paper: SQL tables, Cypher graphs, HNSW vectors, and integrated hybrid queries.

3 UNIFIED DATA & QUERY MODEL

ArcadeDB adopts a unified logical model in which the same underlying entities can be accessed through relational, graph, and vector perspectives, instead of being replicated into separate engines [8, 13]. In our workload, Stack Overflow objects (users, posts, comments, tags, votes) are represented as typed records with shared identities and properties; these records can be queried as SQL tables, connected as graph vertices/edges, and augmented with embedding vectors for semantic retrieval. This model reduces impedance mismatch across modalities because keys, attributes, and relationships refer to the same persistent objects across all query paths.

At the query level, ArcadeDB exposes multiple languages over this shared model. SQL covers relational analytics, filtering, grouping, and joins; OpenCypher supports path-oriented graph matching and traversal patterns [15, 19]; vector operators support nearest-neighbor retrieval using HNSW-style indexing [14]. While these interfaces are syntactically different, they execute over one storage/transaction substrate, allowing applications to combine modalities without external ETL or asynchronous synchronization. This is consistent with broader efforts toward multi-model query unification and composability [1, 18].

For this paper, we operationalize the model with a progressive pipeline and a hybrid query objective. The table view captures canonical relational workloads; the graph view materializes relationship-centric access patterns; and the vector view adds semantic similarity over textual content. Hybrid queries then compose these steps in one workflow, e.g., using vector retrieval to identify semantically relevant posts, graph expansion to propagate through user/tag neighborhoods, and SQL predicates/aggregations to produce final ranked results. This unified model is the semantic foundation for our four-pillar evaluation and for the comparison between integrated execution and polyglot alternatives.

4 EXECUTION & STORAGE ARCHITECTURE

ArcadeDB's execution path is organized around a single embedded runtime that couples storage, transactions, and query processing, in line with a native multi-model architecture perspective [8, 13]. At the lowest layer, data and index structures are persisted as local paginated components, with write-ahead logging (WAL) and configurable flush policies used to provide durability and recovery semantics within one process. The core database instance is thread-safe and exposes explicit transaction lifecycle controls (begin/commit/rollback) together with isolation-level settings, so SQL, Cypher, and vector operations participate in the same transactional context instead of crossing system boundaries.

Query execution is mediated by a language-aware dispatch layer. Commands are submitted with an explicit language identifier and routed through the query-engine manager to the corresponding runtime (e.g., SQL or OpenCypher). SQL and Cypher engines maintain statement/plan caches to reduce repeated parse and planning overhead, while still executing against shared schema metadata, record identifiers, and indexes. This architecture separates language front-ends from storage internals: different syntactic interfaces can optimize their own parsing/planning pipelines without fragmenting persistence or concurrency control.

Graph operations are implemented through a native graph subsystem that manages vertex/edge creation and adjacency maintenance as first-class database structures. Vector retrieval is implemented through a page-based transactional vector index that integrates with the same storage manager and schema layer. The vector stack is configured through schema-level builders (dimensions, similarity function, HNSW parameters, quantization), and supports hybrid scoring functions in SQL to combine lexical and semantic signals [5, 14, 20]. In the context of this paper, the key architectural property is not any single micro-optimization, but the co-location of table, graph, and vector execution paths over one engine state. This co-location is what enables our hybrid pipeline (vector candidate generation, graph expansion, and SQL filtering/aggregation) without external replication or inter-system RPC orchestration.

5 EXPERIMENTAL SETUP

We evaluate ArcadeDB on the Stack Overflow benchmark using seven executable workloads that map directly to our pipeline stages: table OLTP, table OLAP, graph OLTP, graph OLAP, vector index build, vector search, and end-to-end hybrid queries. The reported dataset is a Stack Overflow snapshot (stats.stackexchange.com, 2024-06-30) with approximately 2.9 GB of source data and 5,564,864 records in total, distributed as follows: User (345,754), Post (425,735), Comment (819,648), Badge (612,258), Vote (1,747,225), PostLink (86,919), Tag (1,612), and PostHistory (1,525,713). For vector workloads, the combined corpus contains 1,242,391 vectors and occupies approximately 2.0 GB on disk. We use repeated runs under controlled seeds and consolidated summaries. All measurements are collected on a single host with an AMD Ryzen 9 7950X CPU (16 cores / 32 threads), DDR5 RAM, and a Samsung 970 EVO Plus SSD. For graph workloads, we materialize one fixed property-graph schema from the same source tables. Vertex classes are User, Question, Answer, Tag, Badge, and Comment. Edge classes include asked/answered links, answer ownership and acceptance links, tag assignments, comment links (to questions and answers), badge ownership, and post-link relations. Concretely, Posts are split into Question and Answer vertices via the PostTypeId field, and XML IDs plus foreign keys are used to instantiate edge endpoints consistently across both graph systems.

Baselines are selected per modality. For table workloads, ArcadeDB is compared with SQLite, DuckDB, and PostgreSQL [7, 22, 26]. For graph workloads, ArcadeDB is compared with LadybugDB under a shared query/operation suite [11]. For vector workloads, we compare four backends: ArcadeDB, pgvector, Qdrant, and Milvus [16, 21, 24]. The vector build benchmark uses common HNSW-style controls (e.g., max connections and beam width), while the search benchmark uses matched retrieval-depth settings across systems: `efSearch`=100 for pgvector/Qdrant/Milvus and ArcadeDB over-query factor = 4.0 (ArcadeDB does not expose `efSearch` directly). We then report effectiveness/latency trade-offs under this normalized setup [14, 20]. To keep artifact sharing and downstream reuse straightforward, we restrict compared systems in this study to permissive-license software (e.g., Apache-2.0/BSD/MIT) and therefore do not include engines under non-permissive or source-available commercial terms (such as Neo4j or ArangoDB) in these benchmark matrices [2, 17].

Execution policy follows the protocol used in our matrix scripts and summaries. OLTP results are reported in single-threaded mode for cross-system fairness, while OLAP runs use a multi-threaded CPU budget across all systems in this study. We keep dataset and schema definitions fixed across systems and separate lifecycle stages into load/ingest, index construction, and measured query/transaction execution. Each benchmark emits both per-run JSON artifacts and aggregated markdown summaries, from which we report the core metrics used in the paper: OLTP latency percentiles and throughput, OLAP query runtimes, vector recall/latency curves, and storage plus peak memory footprints. This setup aligns with prior multi-model and holistic benchmarking principles while keeping the comparison grounded in executable, reproducible scripts [10, 31, 32]. To support industrial reproducibility and downstream validation, all implementation code and benchmarking scripts used in this study are maintained in the public ArcadeDB repository under Apache-2.0 licensing [3]. The artifact includes runnable automation assets (e.g., Docker Compose workflows and deployment manifests) so engineering teams can reproduce measurements and adapt them to their own CI/CD and infrastructure constraints.

Table 1: SQL table OLTP (single-threaded, Stack Overflow dataset, mean over 3 runs).

System	Throughput (ops/s)	p50 (ms)	p95 (ms)	p99 (ms)
ArcadeDB	578.5	0.155	8.99	24.90
SQLite	1600.9	0.019	3.40	12.61
PostgreSQL	1169.1	0.098	4.53	13.09
DuckDB	117.7	1.945	22.29	60.23

6 RESULTS

6.1 SQL/Table Workloads

6.1.1 SQL/Table OLTP. Table 1 reports single-threaded OLTP results on the Stack Overflow dataset using 100,000 mixed CRUD transactions (60% read, 20% update, 10% insert, 10% delete). We focus on steady-state transactional metrics (throughput and end-to-end latency percentiles) and exclude preload / ingestion time from this comparison because loading paths are system-specific and not part of the OLTP critical path.

SQLite and PostgreSQL lead this single-threaded OLTP setting, while ArcadeDB is intermediate and DuckDB trails on both throughput and tail latency. This ordering is consistent with engine design goals: SQLite, PostgreSQL, and ArcadeDB are optimized for transactional paths, whereas DuckDB is primarily optimized for analytical (OLAP) execution. Relative to DuckDB, ArcadeDB delivers about 4.9× higher throughput with lower tail latency (about 2.5× lower p95). These results suggest that ArcadeDB remains competitive for embedded mixed-transaction paths, though specialized row-store OLTP engines retain an advantage in this specific table-only regime.

Representative SQL OLTP operation (update):

```
UPDATE Post SET Score = coalesce(Score, 0) + 1 WHERE Id =
:target_id
```

6.1.2 SQL/Table OLAP. Table 2 reports analytical query execution time for the fixed 10-query suite under shuffled order and repeated runs, including both cold and warm per-query means. In this OLAP setting, DuckDB leads by a wide margin, followed by SQLite and PostgreSQL, while ArcadeDB

Table 2: SQL table OLAP execution (Stack Overflow dataset, mean over 3 runs, 10-query suite with 10 shuffled runs).

System	Query total (s)	Cold (ms/q)	Warm (ms/q)
ArcadeDB	433.376	7722.1	3956.9
SQLite	4.906	48.5	49.1
PostgreSQL	6.724	43.6	69.8
DuckDB	0.221	3.9	2.0

Table 3: Graph/Cypher OLTP execution (single-threaded, Stack Overflow dataset, mean over 3 runs).

System	Throughput (ops/s)	p50 (ms)	p95 (ms)	p99 (ms)
ArcadeDB	416.3	0.597	7.75	30.03
Ladybug	51.1	15.01	71.15	92.85

is slower on this specific table-analytics workload. This ordering is consistent with design intent: DuckDB is OLAP-oriented, whereas ArcadeDB is currently optimized more for transactional and multi-model execution paths. Correctness remained stable across systems in this benchmark: query result hashes were stable within each DB and equal across DBs for all reported OLAP queries.

Representative SQL OLAP query:

```
SELECT Id, DisplayName, Reputation FROM User WHERE
Reputation IS NOT NULL ORDER BY Reputation DESC, Id ASC
LIMIT 10
```

6.2 Graph/Cypher Workloads

6.2.1 Graph/Cypher OLTP. Table 3 reports mixed graph OLTP execution (read/update/insert/delete = 60/20/10/10) in single-threaded mode. On these runs, ArcadeDB shows higher throughput and lower tail latency than Ladybug. As in the table-OLTP discussion, we focus on CRUD/query-path execution metrics and not ingest timing, since load paths differ by engine. For practical runtime in this draft, Ladybug was executed at 10% transaction volume (10,000 vs. 100,000 for ArcadeDB) because it is primarily OLAP-oriented and full-volume OLTP runs were too slow.

Representative Cypher OLTP operation (edge-property update):

```
MATCH (u:User {Id: :uid})-[r:ANSWERED]->(a:Answer {Id:
:aid}) SET r.CreationDate = coalesce(r.CreationDate, 0) +
1
```

6.2.2 Graph/Cypher OLAP. Table 4 reports analytical Cypher execution time for the fixed 10-query suite under shuffled order and repeated runs, including both cold and warm per-query means. In this graph-OLAP setting, Ladybug is faster, while ArcadeDB is slower on this specific analytical query

Table 4: Graph/Cypher OLAP execution (Stack Overflow dataset, mean over 3 runs, 10-query suite with 10 shuffled runs).

System	Query total (s)	Cold (ms/q)	Warm (ms/q)
ArcadeDB	302.930	3262.9	3002.9
Ladybug	10.051	115.9	98.3

Table 5: Vector index build on Stack Overflow vectors (1,242,391 vectors, dim=384, HNSW m=16, ef_construction/beam=100, mean over 3 runs).

Backend	Ingest (s)	Index (s)	Total (s)	Mem (GiB)	Size (GiB)
ArcadeDB	44.3	2362.0	2406.8	8.8	2.7
pgvector	171.3	1139.1	1313.2	8.7	5.3
Qdrant	327.7	0.6	331.8	3.3	2.3
Milvus	222.0	0.8	240.1	2.9	8.6

mix. This ordering is consistent with engine focus: Ladybug is optimized for graph analytics, whereas ArcadeDB currently prioritizes transactional and unified multi-model execution paths. Correctness remained stable across both systems in this benchmark: query result hashes and row counts were stable within each DB and equal across DBs for all 10 queries.

Representative Cypher OLAP query:

```
MATCH (q:Question)-[:TAGGED_WITH]->(t1:Tag) MATCH
(q)-[:TAGGED_WITH]->(t2:Tag) WHERE t1.Id < t2.Id RETURN
t1.TagName, t2.TagName, count(*) AS cooccurs ORDER BY
cooccurs DESC LIMIT 10
```

6.3 Vector/HNSW Workloads

6.3.1 Vector Index Build. Table 5 summarizes vector build behavior across four backends on the Stack Overflow embedding corpus. By total wall-clock build time, Milvus and Qdrant are fastest, followed by pgvector, while ArcadeDB is slower in this setting. Phase breakdowns show a backend-specific execution pattern: ArcadeDB and pgvector expose substantial explicit index-build time, whereas Qdrant and Milvus spend most work during ingest with near-zero standalone index phase. Resource trade-offs are also visible: Qdrant has the smallest on-disk footprint, Milvus the largest, and peak memory is lower for Qdrant/Milvus than for ArcadeDB/pgvector on this configuration.

For client-server backends (pgvector, Qdrant, Milvus), reported peak RSS combines client process memory with server process-tree memory, following the benchmark script policy.

Representative vector-index build statement (pgvector):

```
CREATE INDEX vectordata_vector_hnsw ON vectordata USING
hnsw (vector vector_cosine_ops) WITH (m = 16,
ef_construction = 100)
```

6.3.2 Vector Search. Table 6 reports search quality and latency at one normalized operating point across backends. Qdrant achieves the highest Recall@50 in this setting, while Milvus provides the lowest mean and p95 query latency;

Table 6: Vector search on Stack Overflow vectors (k=50, 1,000 queries, mean over 3 runs; normalized depth: ArcadeDB over-query factor=4, pgvector/Qdrant/Milvus `efSearch`=100).

Backend	Recall@50	Mean (ms/q)	p95 (ms/q)	Mem (GiB)
ArcadeDB	0.950	290.4	635.2	6.4
pgvector	0.967	80.8	391.8	2.3
Qdrant	0.989	53.0	56.9	2.5
Milvus	0.972	18.7	33.9	4.3

Table 7: Hybrid query workflows on Stack Overflow (ArcadeDB native pipeline, heap=16g).

Query workflow	Time (s)	Dominant step	Rows
SQL → Vector	4.42	sql_candidates (4.15s)	10
SQL → Cypher	6.04	cypher_expand (6.03s)	10
Cypher → SQL	6.09	cypher_seed_users (5.95s)	10
Vector → Cypher	4.02	cypher_expand (4.02s)	10
Vector → Cypher → SQL	2.91	cypher_expand_users (2.90s)	5

pgvector is intermediate on both quality and latency. ArcadeDB shows lower recall and higher latency at this operating point. As in the build benchmark, resource usage differs by architecture: pgvector has the lowest peak memory here.

The search-depth knobs are aligned by benchmark policy but are not semantically identical (ArcadeDB over-query factor versus HNSW `efSearch`); this comparison should be interpreted as a practical normalized setting rather than parameter-level equivalence. Query-result and row-count stability remained true across repeated runs for all backends.

Representative vector-search query (pgvector path):

```
SET LOCAL hnsw.ef_search = 100; SELECT id FROM vectordata
ORDER BY vector <=> :query_vector LIMIT 50
```

6.4 Hybrid Query Showcase

Table 7 reports end-to-end timings for five cross-model workflows executed natively in one ArcadeDB pipeline. This section emphasizes composability across SQL, Cypher, and vector operators rather than single-model leaderboards. In this run, all workflows complete in 2.9–6.1s once the pipeline is prepared, with graph expansion dominating runtime in most patterns. Q1 differs by being dominated by SQL candidate generation before vector reranking. Build-stage context is important for interpretation: the overall run (3210.0s, peak RSS 11.9GiB) is dominated by embedding and vector-index preparation, especially Comment-vector indexing.

Representative hybrid workflow (Vector → Cypher → SQL):

```
(1) find_nearest(query, k=10, overquery_factor=4)
(2) MATCH (q)-[:HAS_ANSWER]->(a)<-[:ANSWERED]-(u) WHERE
q.Id IN [..] RETURN DISTINCT u.Id
(3) SELECT Id, DisplayName, Reputation, Views FROM User
WHERE Id IN [..] AND Reputation >= 1000 ORDER BY
Reputation DESC, Views DESC LIMIT 10
```

7 DISCUSSION & LIMITATIONS

The results show a consistent pattern across modalities. In single-model leaderboards, specialized engines often remain ahead on their primary target workloads (e.g., DuckDB for table OLAP, Ladybug for graph OLAP, and Qdrant/Milvus for the evaluated vector-search operating point). At the same time, ArcadeDB remains practically competitive in transactional table/graph paths and, most importantly, executes cross-model SQL–Cypher–vector workflows natively in one runtime. This confirms the central trade-off in our study: a unified embedded architecture may not dominate every isolated micro-benchmark, but it can reduce system-composition overhead while preserving usable performance across heterogeneous query classes [8, 13, 31].

From a systems perspective, the hybrid showcase is the key outcome. End-to-end workflows complete in seconds once the pipeline is prepared, and the dominant stage is transparent from per-step timing (often graph expansion, sometimes SQL candidate generation). This observability makes optimization targets explicit for practitioners. More broadly, these runs indicate that integrated execution can shift effort from ETL/RPC orchestration toward in-engine plan and index tuning, which is often easier to reproduce and maintain in embedded deployments [30, 31].

From an industrial decision-making perspective, the reported matrices are intended as engineering guidance rather than a universal ranking: they expose where specialized engines retain clear single-model advantages, and where a unified runtime can reduce integration overhead in cross-model pipelines. This trade-off framing is directly relevant for teams deciding between polyglot stacks and consolidated data-platform architectures under operational constraints.

Several limitations should frame interpretation. First, this paper uses one primary dataset family (Stack Overflow) and one main hardware/runtime profile, so external validity to other graph topologies, vector distributions, or memory budgets is not yet established. Second, OLTP is reported in single-threaded mode in this draft, while OLAP uses multi-threaded execution; this reflects our fairness protocol but does not fully characterize concurrency scaling. Third, the graph OLTP comparison includes reduced Ladybug transaction volume for runtime practicality, and vector search is reported at one normalized depth setting rather than full recall-latency frontiers, which are commonly needed to characterize ANN trade-offs [14, 20]. Finally, baseline scope is intentionally restricted to permissive-license systems for reproducible artifact sharing.

These constraints motivate immediate follow-up work: broader workload families and hardware profiles, explicit multi-thread OLTP scaling studies, full vector operating curves under matched quality targets, and deeper optimization of ArcadeDB's analytical and vector execution paths. We view this paper as a reproducible foundation for that expanded evaluation [10, 31, 32].

8 RELATED WORK

Our study intersects four literature threads: multi-model data management, holistic benchmarking, graph query systems, and vector retrieval engines. Prior work argues that model fragmentation (relational/graph/document/vector) increases integration and consistency overhead, motivating unified or tightly integrated architectures [8, 12, 13, 30]. Our paper follows this line by evaluating one native engine across SQL, Cypher, vector search, and composed hybrid workflows rather than isolating only one interface.

Benchmark methodology for multi-model systems has been discussed in UniBench and related holistic benchmarking work, which emphasize common datasets, controlled workload generation, and cross-model comparability [31, 32]. We align with these principles by using one Stack Overflow pipeline and executable scripts for all stages (table OLTP / OLAP, graph OLTP / OLAP, vector build/search, and hybrid). In contrast to benchmark suites that focus on one model family at a time, we explicitly report trade-offs across all three modalities in one experimental lifecycle [10].

For graph query processing, OpenCypher semantics and formalization provide the language basis for portable graph workloads [15, 19], while prior graph system evaluations highlight engine-specific strengths under traversal- and analytics-heavy patterns [6, 23, 25]. Our graph experiments follow this tradition but are embedded in a broader multi-model comparison, so graph results are interpreted together with SQL and vector behavior rather than as standalone leaderboard outcomes.

For vector retrieval, HNSW and its descendants establish the core recall-latency trade-offs and index-construction design space [4, 14, 27]. Recent systems and surveys describe diverse implementation choices in storage layout, filtering support, and operational tuning [9, 20, 28, 29]. Our vector evaluation adopts this perspective by reporting both effectiveness and resource costs, then connecting them to hybrid SQL/Cypher/vector execution in one engine. This placement differentiates our contribution from pure ANN benchmark papers: we treat vector search as one component of an end-to-end multi-model workflow rather than as an isolated service benchmark.

9 CONCLUSION

This paper presented ArcadeDB as a native embedded multi-model database that executes SQL, Cypher, and vector retrieval over one shared runtime and data representation. Using one Stack Overflow pipeline, we evaluated table OLTP / OLAP, graph OLTP / OLAP, vector index build/search, and hybrid cross-model workflows under a reproducible script-driven setup.

Our results show a clear practical trade-off. Specialized engines often lead on their primary single-model workloads, while ArcadeDB remains competitive in transactional paths and, crucially, supports end-to-end SQL–Cypher–vector composition natively without external ETL or multi-service orchestration. The hybrid experiments demonstrate that this integrated execution model is measurable and operationally meaningful, not only an architectural claim.

At the same time, we identify explicit limits of the current study: one primary dataset family, mixed threading regimes across workload classes, reduced transaction volume for one graph baseline in OLTP, and vector reporting at one normalized operating point. These limits define a concrete next agenda: broader workload diversity, explicit concurrency-scaling analysis, and full recall-latency frontier studies under matched quality targets. We hope this work serves as a reproducible baseline for evaluating unified multi-model embedded systems in realistic application pipelines.

REFERENCES

[1] 2023. ISO/IEC 9075-16:2023 Information technology — Database languages SQL — Part 16: Property Graph Queries (SQL/PGQ). https://www.iso.org/standard/79473.html First edition, accessed 2026-02-25.

[2] ArangoDB GmbH. 2026. ArangoDB Documentation. https://docs.arangodb.com/ Accessed: 2026-02-25.

[3] Arcade Data Ltd. 2026. ArcadeDB GitHub Repository. https://github.com/ArcadeData/arcadedb Apache-2.0 licensed open-source project; Accessed: 2026-02-26.

[4] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighbor Search. https://doi.org/10.48550/arXiv.2111.08566 arXiv:2111.08566 [cs.DB]

[5] Yannis Chronis, Helena Caminal, Yannis Papakonstantinou, Fatma "Ozcan, and Anastasia Ailamaki. 2025. Filtered Vector Search: State-of-the-Art and Research Opportunities. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5488–5492. https://doi.org/10.14778/3750601.3750700

[6] Orri Erling, Alex Averbuch, Josep Larriba-Pey, Hassan Chafi, Andrey Gubichev, Arnau Prat, Minh-Duc Pham, and Peter Boncz. 2015. The LDBC Social Network Benchmark: Interactive Workload. In *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. 619–630. https://doi.org/10.1145/2723372.2742786

[7] Kevin P. Gaffney, Martin Prammer, Larry Brasfield, D. Richard Hipp, Dan Kennedy, and Jignesh M. Patel. 2022. SQLite: past, present, and future. *Proceedings of the VLDB Endowment* 15, 12 (2022), 3535–3547. https://doi.org/10.14778/3554821.3554842

[8] Qingsong Guo, Chao Zhang, Shuxun Zhang, and Jiaheng Lu. 2023. Multi-model query languages: taming the variety of big data. *Distributed and Parallel Databases* 42, 1 (2023), 31–71. https://doi.org/10.1007/s10619-023-07433-1

[9] Guoyu Hu, Shaofeng Cai, Tien Tuan Anh Dinh, Zhongle Xie, Cong Yue, Gang Chen, and Beng Chin Ooi. 2025. HAKES: Scalable Vector Database for Embedding Search Service. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3049–3062. https://doi.org/10.14778/3746405.3746427

[10] Bogyeong Kim, Kyoseung Koo, Undraa Enkhbat, Sohyun Kim, Juhun Kim, and Bongki Moon. 2022. M2Bench: A Database Benchmark for Multi-Model Analytic Workloads. *Proceedings of the VLDB Endowment* 16, 4 (2022), 747–759. https://doi.org/10.14778/3574245.3574259

[11] LadybugDB Developers. 2026. Ladybug Documentation. https://docs.ladybugdb.com/ Accessed: 2026-02-25.

[12] Hongyu Lei, Chunhua Li, Ke Zhou, Jianping Zhu, Kezhou Yan, Fen Xiao, Ming Xie, Jiang Wang, and Shiyu Di. 2024. X-Stor: A Cloud-Native NoSQL Database Service with Multi-Model Support. *Proceedings of the VLDB Endowment* 17, 12 (2024), 4025–4037. https://doi.org/10.14778/3685800.3685824

[13] Jiaheng Lu and Irena Holubová. 2019. Multi-model Databases: A New Journey to Handle the Variety of Data. *Comput. Surveys* 52, 3 (2019), 1–38. https://doi.org/10.1145/3323214

[14] Yu. A. Malkov and D. A. Yashunin. 2016. Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs. https://doi.org/10.48550/arXiv.1603.09320 arXiv:1603.09320 [cs.DS]

[15] J'ozsef Marton, G'abor Sz'arnyas, and D'aniel Varr'o. 2017. Formalising opencypher Graph Queries in Relational Algebra. https://doi.org/10.48550/arXiv.1705.02844 arXiv:1705.02844 [cs.DB]

[16] Milvus Developers. 2026. Milvus Documentation. https://milvus.io/docs Accessed: 2026-02-25.

[17] Neo4j, Inc. 2026. Neo4j Documentation. https://neo4j.com/docs/ Accessed: 2026-02-25.

[18] Kian Win Ong, Yannis Papakonstantinou, and Romain Vernoux. 2014. The SQL++ Query Language: Configurable, Unifying and Semi-structured. https://doi.org/10.48550/arXiv.1405.3631 arXiv:1405.3631 [cs.DB]

[19] openCypher Implementers Group. 2025. openCypher 9 Specification and Resources. https://opencypher.org/ Accessed: 2026-02-25.

[20] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *The VLDB Journal* 33, 5 (2024), 1591–1615. https://doi.org/10.1007/s00778-024-00864-x

[21] pgvector contributors. 2026. pgvector: Open-source vector similarity search for Postgres. https://github.com/pgvector/pgvector Accessed: 2026-02-25.

[22] PostgreSQL Global Development Group. 2026. PostgreSQL Documentation. https://www.postgresql.org/docs/ Accessed: 2026-02-25.

[23] David P"uroja, Jack Waudby, Peter Boncz, and G'abor Sz'arnyas. 2024. *The LDBC Social Network Benchmark Interactive Workload v2: A Transactional Graph Query Benchmark with Deep Delete Operations*. 107–123. https://doi.org/10.1007/978-3-031-68031-1_8

[24] Qdrant Team. 2026. Qdrant Documentation. https://qdrant.tech/documentation/ Accessed: 2026-02-25.

[25] Shipeng Qi, Bing Tong, Jiatao Hu, Heng Lin, Yue Pang, Wei Yuan, Songlin Lyu, Zhihui Guo, Ke Huang, Xujin Ba, Qiang Yin, Youren Shen, Yan Zhou, Tao Lv, Jia Li, Lei Zou, Yongwei Wu, G'abor Sz'arnyas, Xiaowei Zhu, Wenguang Chen, and Chuntao Hong. 2025. The LDBC Financial Benchmark: Transaction Workload. *Proceedings of the VLDB Endowment* 18, 9 (2025), 3007–3020. https://doi.org/10.14778/3746405.3746424

[26] Mark Raasveldt and Hannes M"uhleisen. 2019. DuckDB: an Embeddable Analytical Database. In *Proceedings of the 2019 International Conference on Management of Data*. 1981–1984. https://doi.org/10.1145/3299869.3320212

[27] Aditi Singh, Suhas Jayaram Subramanya, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2021. FreshDiskANN: A Fast and Accurate Graph-Based ANN Index for Streaming Similarity Search. https://doi.org/10.48550/arXiv.2105.09613 arXiv:2105.09613 [cs.IR]

[28] Ji Sun, Guoliang Li, James Pan, Jiang Wang, Yongqing Xie, Ruicheng Liu, and Wen Nie. 2025. GaussDB-Vector: A Large-Scale Persistent Real-Time Vector Database for LLM Applications. *Proceedings of the VLDB Endowment* 18, 12 (2025), 4951–4963. https://doi.org/10.14778/3750601.3750619

[29] Nitish Upreti, Harsha Vardhan Simhadri, Hari Sudan Sundar, Krishnan Sundaram, Samer Boshra, Balachandar Perumalswamy, Shivam Atri, Martin Chisholm, Revti Raman Singh, Greg Yang, Tamara Hass, Nitesh Dudhey, Subramanyam Pattipaka, Mark Hildebrand, Magdalen Manohar, Jack Moffitt, Haiyang Xu, Naren Datha, Suryansh Gupta, Ravishankar Krishnaswamy, Prashant Gupta, Abhishek Sahu, Hemeswari Varada, Sudhanshu Barthwal, Ritika Mor, James Codella, Shaun Cooper, Kevin Pilch, Simon Moreno, Aayush Kataria, Santosh Kulkarni, Neil Deshpande, Amar Sagare, Dinesh Billa, Zishan Fu, and Vipul Vishal. 2025. Cost-Effective, Low Latency Vector Search with Azure Cosmos DB. *Proceedings of the VLDB Endowment* 18, 12 (2025), 5166–5183. https://doi.org/10.14778/3750601.3750635

[30] Feng Ye, Xinjun Sheng, Nadia Nedjah, Jun Sun, and Peng Zhang. 2023. A Benchmark for Performance Evaluation of a Multi-Model Database vs. Polyglot Persistence. *Journal of Database Management* 34, 3 (2023), 1–20. https://doi.org/10.4018/JDM.321756

[31] Chao Zhang and Jiaheng Lu. 2019. Holistic evaluation in multi-model databases benchmarking. *Distributed and Parallel Databases* 39, 1 (2019), 1–33. https://doi.org/10.1007/s10619-019-07279-6

[32] Chao Zhang, Jiaheng Lu, Pengfei Xu, and Yuxing Chen. 2019. *UniBench: A Benchmark for Multi-model Database Management Systems*. 7–23. https://doi.org/10.1007/978-3-030-11404-6_2